

HR Employee Attrition Analysis

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# =====
# STEP 1: Load Data & Quick Inspection
# =====
# Replace 'hr_data.csv' with your actual dataset path
# For demonstration, we assume a standard HR analytics format
try:
    df = pd.read_csv('hr_data.csv')
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("Please provide a valid path to your HR dataset CSV.")
# Creating a dummy dataframe for code execution continuity if run locally
np.random.seed(42)
df = pd.DataFrame({
    'Age': np.random.randint(22, 60, 500),
    'Attrition': np.random.choice(['Yes', 'No'], 500, p=[0.16, 0.84]),
    'MonthlyIncome': np.random.randint(3000, 15000, 500),
    'JobSatisfaction': np.random.randint(1, 5, 500),
```

```

        'WorkLifeBalance': np.random.randint(1, 5, 500),
        'YearsAtCompany': np.random.randint(1, 15, 500),
        'OverTime': np.random.choice(['Yes', 'No'], 500)
    })

print(df.head())

print("\nMissing Values:\n", df.isnull().sum())

# =====

# STEP 2: Exploratory Data Analysis (EDA)

# =====

plt.figure(figsize=(6,4))
sns.countplot(x='Attrition', data=df, palette='Set2')
plt.title('Overall Attrition Count')
plt.show()

# Income vs Attrition

plt.figure(figsize=(6,4))
sns.boxplot(x='Attrition', y='MonthlyIncome', data=df, palette='Pastel1')
plt.title('Monthly Income vs Attrition')
plt.show()

# Overtime vs Attrition

plt.figure(figsize=(6,4))
sns.countplot(x='OverTime', hue='Attrition', data=df, palette='viridis')
plt.title('Overtime Impact on Attrition')
plt.show()

# =====

```

```

# STEP 3: Data Preprocessing

# =====

# Convert target variable 'Attrition' to numeric (No: 0, Yes: 1)

le = LabelEncoder()

df['Attrition'] = le.fit_transform(df['Attrition'])


# Encode other categorical variables (e.g., OverTime)

if 'OverTime' in df.columns:

    df['OverTime'] = le.fit_transform(df['OverTime'])


# If your dataset has more text columns (like Department, BusinessTravel),
# use pd.get_dummies(df) to one-hot encode them.

df_encoded = pd.get_dummies(df, drop_first=True)


# Split features and target

X = df_encoded.drop('Attrition', axis=1)
y = df_encoded['Attrition']


# Split into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)


# Feature Scaling

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# =====

# STEP 4: Model Training (Random Forest)

# =====

```

```

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train_scaled, y_train)

# Predictions
y_pred = model.predict(X_test_scaled)

# =====
# STEP 5: Model Evaluation
# =====

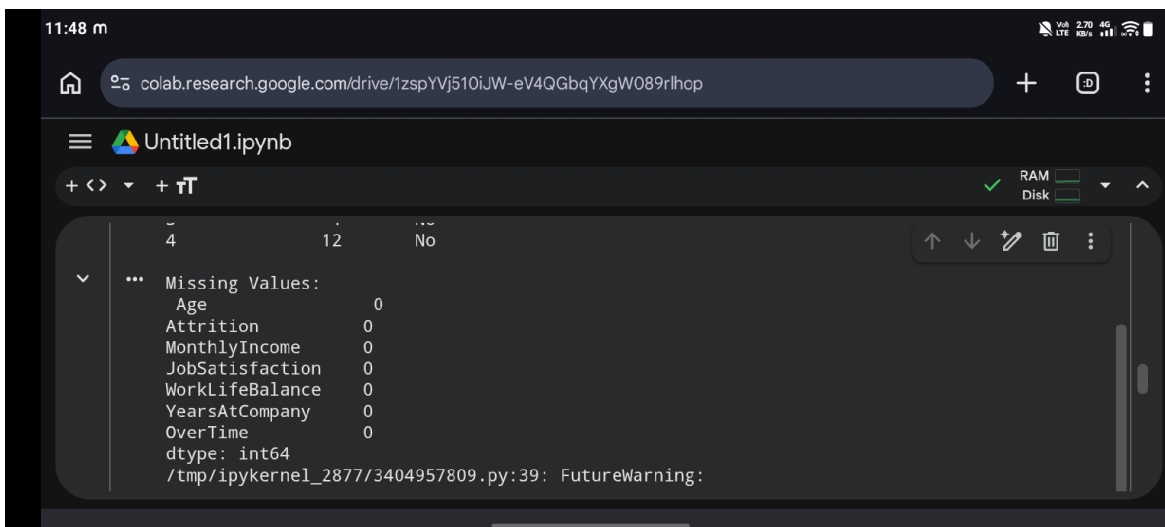
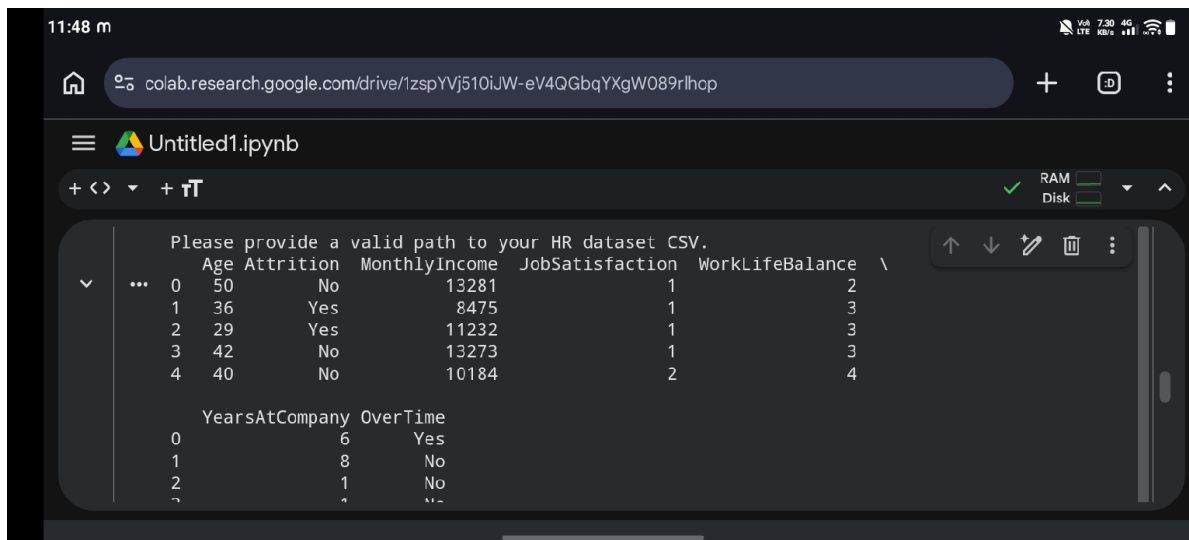
print("\n--- Model Performance Evaluation ---")
print(f"Accuracy Score: {accuracy_score(y_test, y_pred):.2f}")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Feature Importance
importances = model.feature_importances_
feature_names = X.columns
model_importances = pd.Series(importances, index=feature_names).sort_values(ascending=False)

plt.figure(figsize=(8,5))
sns.barplot(x=model_importances.values, y=model_importances.index, palette='mako')
plt.title('Top Factors Driving Employee Attrition')
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.show()

```

Output :



11:49 m

Vol 4.00 5G+
LTE KB/s

colab.research.google.com/driv



+ <> + T



RAM

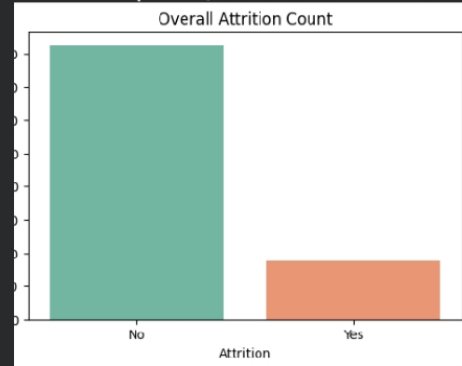
Disk



using palette



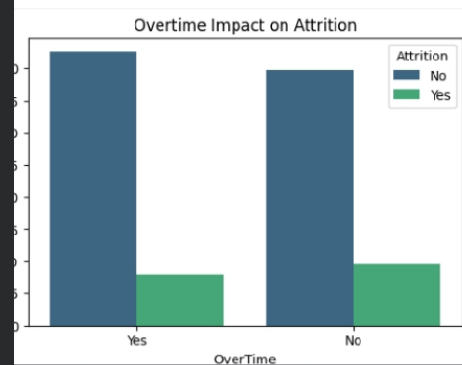
```
ns.countplot(x='Attrition', data=c
```



p/ipykernel_2877/3404957809.py:45:

using `palette` without assigning

```
ns.boxplot(x='Attrition', y='Month
```





colab.research.google.com/driv







RAM 
Disk 



0

Yes



...

-- Model Performance Evaluation --
Accuracy Score: 0.80

Confusion Matrix:
[[80 2]
 [18 0]]

Classification Report:

	precision	recall
0	0.82	0.98
1	0.00	0.00

accuracy		
macro avg	0.41	0.49
weighted avg	0.67	0.80

tmp/ipykernel_2877/3404957809.py:1
assing `palette` without assigning

sns.barplot(x=model_importances.v

Top Factors Driving Employee Attrition

Monthlyincome 



colab.research.google.com/driv



RAM



Disk



accuracy

macro avg

weighted avg

0.67

0.80



tmp/ipykernel_2877/3404957809.py:1

assing `palette` without assigning

sns.barplot(x=model_importances.v

